ownership_repair

`scripts/ownership_repair.sh` — Ownership Repair

Revision: 7 **Last modified:** 2026-08-27T00:00:00Z **Purpose:** Operator guide for the tool that brings pre-existing content back under the ownership of the person who started the system. **Last verified:** 2026-08-27

Overview

`scripts/ownership_repair.sh` walks every location declared in `config/owned_paths.yaml` and `chown`s every item that is **not** owned by the operator back to the operator's `uid:gid`.

"Owned by the operator" means the `uid` matches — nothing else. The walk selects on `uid` alone, and an item the operator already owns is left untouched even if its *group* differs. That is the feature's single definition of the property: data-model E4 models the ownership probe with `probe_uid`/`expected_uid` and no gid field, `contracts/startup-precondition.md` compares against "the operator's uid", and FR-001/FR-002/FR-003/FR-010b all say "owned by the account". Operationally it is sufficient because owner-class permission bits are selected by `uid` match, so renaming, moving and deleting (FR-003) work whatever the group is.

The `chown` it performs still **writes** `uid:gid` — one syscall, and it gives a genuinely-broken `100999:100999` item a resolvable group. Until BOB-207 the walk also *selected* on gid, which made this script the only place in the feature that meant `uid+gid` while the startup precondition meant `uid`: the precondition could certify a location this walk simultaneously reported as broken. Narrowing the selection settled that, and has a second effect worth knowing — a deliberate shared-group or `setgid` arrangement on content you already own is no longer silently rewritten by a repair that was never asked to touch it.

It exists because of a measured state, not a hypothetical one: on 2026-08-21 the download root contained an item owned by uid `100999` and `config/` contained **51** of them — a rootless-container sub-uid with no host account. The operator could not edit their own configuration, and `config/boba.db` (mode `600`, owner unresolvable) could not be read by the operator at all, which made the backup procedure that `docs/BOBA_DATABASE.md` § 3 *mandates* impossible to perform.

The precondition (`scripts/ownership_precondition.sh`) **detects** that state. This script **fixes** it.

Four behaviours a reader will otherwise get wrong

These are the non-obvious parts. Each is a deliberate design decision with a failure mode behind it.

1. The completion marker is written ONLY on success — so an interrupted run resumes rather than being skipped forever. `logs/ownership/repair-marker.json` is the **last** act of a fully successful pass. Writing it at *start* would let a single interruption mark the repair "done" permanently and silently skip everything it had not reached yet — the run-once optimisation would defeat the repair it optimises. Interrupted ⇒ marker absent ⇒ the next run walks the whole scope again. This is also the operator's escape hatch from a long run: stopping it mid-repair is safe.

2. The marker carries a scope FINGERPRINT — so changing `config/owned_paths.yaml` re-arms the repair. A marker keyed on mere existence would report "already done" about work that was never performed the moment a **new** path is declared: the newly-declared location would be silently never repaired. The marker instead stores the sha256 of the sorted declared scope (`ownership_scope_fingerprint`), and a normal run is a no-op only when the marker's fingerprint matches the scope as it is *right now*. Add, remove, or edit an entry and the next run repairs the new scope.

3. `preserve_mode` guards MODE BITS, not ownership — such an entry is still chowned. `preserve_mode: true` on `config/boba.db` does **not** exempt the credential store from ownership repair; it is repaired like everything else. What `preserve_mode` protects is its **permission bits**. `chown(2)` performed by a non-root user clears setuid/setgid, so the script reads each item's original mode before the change and restores it afterwards. Bringing a credential store under the operator's ownership while widening who can read it would trade a usability defect for a security one — strictly worse than the defect being fixed (FR-015). Mode restoration also fires, `preserve_mode` or not, for any item that actually carried special bits (a 4-digit octal mode).

4. `preserve_mode` entries are processed FIRST — and that ordering is load-bearing. The real scope nests: `config/boba.db` (`preserve_mode: true`) lives inside `config/` (`preserve_mode: false`). Whichever entry reaches an item first is the one whose rules apply, and once repaired the item no longer matches the "wrongly-owned" filter, so the later entry never sees it. Ordering the strictest constraint first makes the strictest rule win, and yields exactly one change-record entry per altered item with no cross-entry deduplication pass.

Prerequisites

- `bash` 4+ and coreutils (`find`, `chown`, `chmod`, `mktemp`, `sha256sum`, `date`, `tr`, `wc`)
- `python3` with **PyYAML** — used by the shared scope parser
- `scripts/lib/ownership.sh` — sourced, never re-implemented
- `config/owned_paths.yaml` — the declared scope (data-model **E1**)
- Write access to `logs/ownership/` under the project root (created on demand)
- **Optional:** `podman` or `docker`, only for the namespace fallback described under *Internal behaviour*
- **Optional:** `nice` and `ionice`, for the host-safety re-exec

Usage examples

Example 1 — the normal invocation

```
scripts/ownership_repair.sh
```

Repairs the declared scope. If a valid marker already exists for the current scope fingerprint, this is a no-op and prints `already complete for this scope (marker: logs/ownership/repair-marker.json) — nothing to do.`

Example 2 — preview without changing anything

```
scripts/ownership_repair.sh --dry-run
```

Prints one `would chown <uid>:<gid> (was <uid>:<gid>) <path>` line per item that would change. Creates **nothing** — not the state directory, not the change record, not the marker. A dry run that left a new directory behind would already have changed the tree it claimed only to report on (measured 2026-08-21: an earlier revision created `logs/ownership/` under `--dry-run`, which the unit suite does not observe).

`--dry-run` also **bypasses the marker check**, so it previews the scope even when a valid marker says the repair is complete.

Example 3 — re-walk despite a valid marker

```
scripts/ownership_repair.sh --force
```

Ignores the marker and walks the scope again. Idempotent: items already correct are never matched by the walk, so a forced run on a clean tree changes nothing and writes an empty change record.

Example 4 — repair a different scope

```
scripts/ownership_repair.sh --scope /path/to/
        other_owned_paths.yaml
scripts/ownership_repair.sh --scope=/path/to/
        other_owned_paths.yaml   # equivalent
```

Both forms are accepted; the value is exported as `OWNED_PATHS_FILE` for the shared library.

Example 5 — combine preview with an alternate scope

```
scripts/ownership_repair.sh --dry-run --scope config/
        owned_paths.yaml
```

Example 6 — suppress the namespace fallback

```
CONTAINER_RUNTIME= scripts/ownership_repair.sh --dry-run
```

An explicitly **empty** `CONTAINER_RUNTIME` means “no container runtime is available” and disables the `<runtime> unshare` fallback entirely. This is how gates and test suites run the script without touching a runtime.

Flags

Flag	Effect
`--force`	Ignore a valid completion marker and re-walk the declared scope.
`--dry-run`	Report what would change; change nothing, record nothing, create nothing. Also bypasses the marker check.
`--scope <path>` / `--scope=<path>`	

Flag	Effect
	Override the declared scope file. A missing argument to the space-separated form is exit `2`.
`--state-dir <p>` / `--state-dir=<p>`	Override where the completion marker and change record live (default `logs/ownership/` under the project root). Use it for any ad-hoc invocation so the run does not share — or overwrite — the live operator's marker and recovery trail. A missing argument to the space-separated form is exit `2`.
`-h`, `--help`	Print usage and exit `0`.
(anything else)	Unrecognised: error + usage to stderr, exit `2`.

Env vars

Variable	Meaning
`OWNED_PATHS_FILE`	Scope file path when `--scope` is absent. Default `config/owned_paths.yaml` under the project root.
`CONTAINER_RUNTIME`	**Set-but-empty** = no runtime available; the `unshare` fallback is disabled. **Unset** = detect `podman`, then `docker`. **Non-empty** = use that command verbatim. Set-but-empty is honoured rather than re-detected: probing behind a value the caller explicitly set would do work the caller said not to do (§11.4.201).
`OWNERSHIP_STATE_DIR`	Where the marker and change record live. Default `<project-root>/logs/ownership`. `--state-dir` sets it. Empty is treated as unset (the default is used) rather than as "the current directory" — silently writing the operator's trail into `$PWD` would be worse than either.
`OWNERSHIP_REPAIR_RENICED`	Internal. Set by the script's own host-safety re-exec so it

Variable	Meaning
	happens exactly once; not intended to be set by hand.
`PYTHON_BIN`	Consulted first when the shared library chooses a PyYAML-capable interpreter.
`TMPDIR`	Where the run's scratch directory is created (removed on exit).
scope placeholders	`${VAR:-default}` inside a declared path is expanded against the live environment — `QBITTORRENT_DATA_DIR` resolves the host-specific download root at run time.

Exit codes

Code	Meaning
`0`	Every in-scope item is operator-owned — repaired now, or already correct. The marker is written.
`1`	At least one item could not be repaired. Each is named individually. **No marker is written**, so the next run retries the whole declared scope.
`2`	Could not run: the scope is missing/unparseable, **the scope declares zero locations**, **a declared entry resolved to no usable location** (see *Why an empty scope is `2` and not `0`* below), **the scope declares a path this repair must never walk** (see *The declared-path fence* below), the fingerprint cannot be computed, the state directory cannot be created, or the change record cannot be opened for append. Nothing was touched.

Why an empty scope is `2` and not `0`

A scope file that reads cleanly but yields **no** locations walked nothing and therefore asserted nothing. Before Revision 3 that produced exit `0` and a completion marker carrying the fingerprint of the empty string (`e3b0c442…b855`), which `start.sh` then read as "already repaired" — so the repair was skipped on every subsequent start. Three real shapes produce it:

```yaml
paths: []            # an explicitly empty list

path:                # the top-level key mistyped — valid YAML,
        parses, yields nothing
  - path: "config"

paths:
  - path: "${QBITTORRENT_DATA_DIR}"   # …with the variable unset,
        expands to nothing
```

All three now exit `2`. `scripts/ownership_precondition.sh` has always refused them the same way; the two consumers of one scope file now agree.

…and so does a scope that only *partly* vanishes (Revision 4)

The rule above covered the case where **every** entry vanished. Measured 2026-08-26, its partial sibling did not: a two-entry scope whose first entry was `${UNSET_VAR}` announced `(1 declared locations)`, walked only the survivor, exited `0` and wrote the marker. One **declared** location had silently disappeared, and no consumer could tell that scope from an honest one-entry one.

Since Revision 4 the scope reader itself refuses, so all three consumers (`ownership_repair.sh`, `ownership_precondition.sh` and the pre-build gate) inherit one rule rather than three crosschecks that could disagree. Two shapes are refused:

```yaml
paths:
  - path: "${UNSET_VAR}"          # expands to nothing — no ':-'
        default supplied
  - path: "/srv/media/downloads"  # …and this one is NOT quietly
        repaired alone

paths:
  - path: "/srv/${UNSET_VAR}/downloads"   # expands to /srv/
        downloads —
                                          # a DIFFERENT path than
        declared
```

The second shape is the sharper one: the path stays non-empty, so the fence's depth floor cannot catch it, and the repair chowned a tree the scope never named.

The whole run is refused, not just the offending entry. The completion marker's fingerprint claims *every* declared location and `start.sh` reads it as "already repaired", so a partial walk that exited `0` would latch the miss on every later start. It is also the decision the declared-path fence already takes for a refused entry — one question, one answer.

`${VAR:-default}` — including the explicit `${VAR:-}` — is you *stating* what empty means and is never refused. That is the shipped scope's first entry (`${QBITTORRENT_DATA_DIR:-/mnt/DATA}`), so a rule that swallowed it would refuse the live configuration. The refusal names the entry, the raw spelling and the variable:

```
ownership: 1 declared entry in config/owned_paths.yaml resolved to
no usable location:
ownership:   entry 1 — path: '${QBITTORRENT_DATA_DIR}' — it
expanded to an empty path:
             ${QBITTORRENT_DATA_DIR} is unset or empty and the
entry supplies no ':-' default
ownership: remedy: give the entry a default (${VAR:-/path}), set
the variable, or remove
             the entry. `optional: true` does not cover this — it
says the path may be
             ABSENT, not that the DECLARATION may be.
```

Note the last line: `optional: true` declares that the path may be missing from the filesystem. It does not license a declaration that denotes nothing.

Two interpolation forms are supported — any other `${…}` spelling is refused

A declared path may interpolate `${VAR}` and `${VAR:-default}` (the explicit `${VAR:-}` included). **Those two forms are the whole of what is interpolated**, and any *other* `${…}` spelling is refused rather than substituted, because the substitution would otherwise leave a marker in the walked path:

spelling	why it is refused
`${A:-${B}}` — one form nested inside another's default	the default stops at the **inner** `}`, so that `}` closes the outer form and the trailing one is stranded. Measured: with both unset the path came out as the literal `…/${B}/leaf`; with `A` set, as the corrupted `…//tmp/x}/leaf`. Neither is what you declared.
`${}`, `${1}`, an unterminated `${VAR`	the substitution cannot consume them at all, so the `${`

spelling	why it is refused
	survives into the path the walk would use.

This is judged on the **declared spelling only** — never on a variable's *value*. A value that happens to contain `${`, or a directory whose name contains a literal `}`, is yours to choose and still resolves normally; both are pinned as golden-FALSE cases in the suite so the rule can never grow into them.

A bare `$NAME` is not interpolation — it is a literal path component. Only `${` openers are judged, so `path: /data/$USER/downloads` is neither expanded nor refused: it denotes a directory whose name really is the six characters `$USER`. Measured 2026-08-26 — a non-optional entry of that shape fails honestly when the directory is absent (`declared path does not exist and is not optional`, exit `1`), but an `optional: true` one logs `absent, declared optional — skipped` and exits **`0`**, which is the same optional-escape the refusals above exist to close, one grammar step away. The skip line prints the literal path, so it is visible in the log rather than silent. Unless you really do mean a `$`-named directory, read a bare `$` in a declared path as a typo for `${…}` — write `${USER}` or `${USER:-default}`.

Refusing a bare `$` in code would be **wrong**, and is deliberately not done: `$`-named directories are real and in use. A Windows-formatted download disk carries a literal `$RECYCLE.BIN`, and such an entry is walked normally today (measured on a real directory of that name: `0/0 items need repair`, exit `0`). A rule that refused it would be a §11.4.201(1) false-positive refusal against a working configuration — which this project treats as exactly as serious as a false pass, and which the nine golden-FALSE assertions in the suite exist to prevent this rule from growing into.

The refusal matters most for an `optional: true` entry. Without this rule a non-optional entry of either shape still failed honestly (`does not exist`, exit `1`), but an optional one logged `absent, declared optional — skipped`, exited **`0`** and wrote the completion marker — a corrupted path that read as a successful run while repairing nothing, and latched that miss on every later start. The message names the entry and the spelling:

```
ownership: 1 declared entry in config/owned_paths.yaml resolved to
no usable location:
ownership:   entry 1 — path: '/srv/${A:-${B}}/leaf' — the entry's
spelling was not fully
             resolved: ${A:-${B} nests one `${...}` inside
another's default, which the
             documented ${VAR} / ${VAR:-default} grammar does not
cover — the inner `}`
             closes the outer form and the remainder is left
```

```
stranded, so the walked path
            would not be the declared one
```

The remedy is the same as for any unresolved entry: give it a default, set the variable, or split the nesting into a single `${VAR:-default}`.

The declared-path fence

The walk only ever names items *under* a declared path — but until Revision 3 nothing constrained what a declared path could **be**, and the shipped scope declares `${QBITTORRENT_DATA_DIR:-/mnt/DATA}`, whose value comes from `.env`. Measured 2026-08-25: with `QBITTORRENT_DATA_DIR=/` the repair really did start `find / \( ! -uid … \)` — a recursive walk of the whole filesystem.

A declared path is now checked before anything is walked:

Entry written as	Rule
repo-relative (`config`, `.env`, `tmp`)	after `..`/`.`/`//` are resolved it must still be **inside the project root**
absolute (`/mnt/DATA`, `${QBITTORRENT_DATA_DIR}`)	at least **2 path components**, and never a system tree (`/ bin /boot /dev /etc /lib / lib32 /lib64 /libx32 /proc / root /sbin /sys /usr /var`) nor anything under one
absolute, *computed* deny (Revision 4)	never the container runtime's own storage — `${XDG_DATA_HOME:-$HOME/.local/share}/containers` — whose files are owned by mapped subuids **by design**

The floor is 2 because the shipped default `/mnt/DATA` has exactly 2 — it is the depth the shipped configuration forces, not a number picked by taste. The denylist is needed as well as the floor: `/usr/lib` has 2 components and clears the floor. `/run`, `/home`, `/media`, `/mnt`, `/opt`, `/srv` and `/tmp` are deliberately **not** denylisted — this host's real library is `/run/media/<user>/<disk>/Downloads`, and refusing it would be a false alarm about a correct configuration.

One refused entry refuses the whole run (exit `2`, nothing touched). A scope naming such a path is not one the repair trusts in part.

What the fence does not do (§11.4.6): it bounds the *shape* of a declared path. It cannot decide that a well-shaped absolute path is the tree you meant — an absolute path outside the project with

enough depth is exactly what a download root is, so it is accepted. It also does not require the path to be operator-owned, because a root-owned mount point with operator-owned content beneath it is the ordinary state of a removable disk.

The container-storage deny exists because `/home/<user>` clears the floor and is deliberately not denylisted, so without it a declared root at or above the rootless image store would be accepted by shape — and the `podman unshare` fallback would rewrite the storage the rootless runtime depends on. Its own limit: it resolves the default and `XDG_DATA_HOME`-relocated graphroot (pure string operations), and **not** one relocated in `storage.conf` or via `CONTAINERS_STORAGE_CONF`, because reading those would make the fence depend on filesystem state at check time. A degenerate computed value (relative, or too shallow because `HOME` was unset) is discarded rather than used, so the deny can never broaden into a top-level prefix.

And it judges the spelling, not the resolved path. Normalisation is lexical — deliberately, so it cannot be raced and can judge a path that does not exist yet — so an existing symlink in a **non-final** component of a declared path is invisible to it, and the walk names items under the link's target. Measured, and with no race required. A symlinked **final** component *is* contained (`find`'s default `-P` does not descend it, and the trailing slash that would defeat that is stripped). What bounds the intermediate case is who can write those components: they sit above the declared root, outside every container bind mount, and an actor able to write there can edit the untracked `.env` that supplies the root anyway — so no capability is gained. Tracked as BOB-159.

If you see `REFUSED <path> — …`, fix `config/owned_paths.yaml` or the environment variable it reads; the message names the resolved path and the rule.

Two signal exits are also possible and are deliberate: `130` on `SIGINT` and `143` on `SIGTERM`, both printing `no marker written; the next run resumes.`

A `--dry-run` exits with the same `RC` it would have produced for real — so a preview that names a declared path which cannot be repaired exits `1`, not `0`. Exiting `0` there would make `--dry-run` useless as a pre-flight check.

Edge cases

- **Absent + `optional: true`** → logged as `absent, declared optional — skipped`, and recorded with `outcome: "skipped"` and every ownership field `null`. `config/boba.db` is exactly this shape before first boot.

- **Absent + not optional** → `FAILED … declared path does not exist and is not optional`, recorded with `outcome: "failed"`, exit `1`. Per E1 an absent non-optional path is an error, not a skip.
- **A directory the walk cannot fully read** → `find` reports both what it could read *and* a non-zero status; both halves are honoured. The readable part is repaired, the unreadable part is a named failure line, and the run exits `1`. It is never a silent gap.
- **A hardlink inside the scope** → `chown(2)` acts on the **inode**, so if an in-scope name and an out-of-scope name share one inode, repairing the in-scope name changes the out-of-scope file's owner too. Measured: `1000:10` → `1000:1000`. The reach is bounded — hardlinks cannot cross filesystems, it needs a writer inside the scope able to create the link, it moves ownership only (never content), and the new owner is always *you*. It is **not** fenced, deliberately: refusing every item with more than one link would refuse the ordinary case, because hardlinking is how a torrent client and a media manager share one payload between the download tree and the library.
- **A declared root that cannot be chowned** → reported with an explicit note that the failing item is the *declared root of the entry*, not a file inside it. The usual cause is a mount point owned by another identity, and the remedy is the mount, not the files. The run still exits `1` and writes no marker, so the whole scope is re-walked next start — that is deliberate, but it means an unresolved mount point makes every start re-walk the library.
- **A mode restore that fails** → the item's ownership *was* repaired, but the FR-015 bit-restore did not happen. It is named (`FAILED … mode … could not be restored (FR-015): <reason>`), recorded with `outcome: "mode-restore-failed"`, and the run exits `1` with no marker. The direction is narrowing-only, so it is never an access *widening* — but "the exact bits are preserved" is a promise, and an unkept promise is not reported as success.
- **A chown that fails** → the reason `chown` (and the namespace fallback) gave is printed with the failing path, so `EPERM`, `EROFS` and `ENOENT` — three different remedies — are distinguishable instead of collapsing into one sentence.
- **A symlink inside the scope** → `chown -h` acts on the link itself, never on its target. Without `-h`, a symlink pointing anywhere on the filesystem would let the repair mutate a file **outside** the declared scope; FR-005 requires out-of-scope reach to be impossible by construction, not merely unintended.
- **Content at a rootless sub-uid (e.g. 100999)** → an unprivileged `chown` cannot touch it. The script falls back to `<runtime> unshare chown -h 0:0`, which re-enters the same user namespace where the host operator's uid maps to uid 0. The fallback runs **only after** a plain `chown` has already failed, so a run that does not need it never invokes the runtime at all.
- **Neither `chown` nor the fallback works for an item** → the item is named on stderr, a corrective `outcome: "failed"` record

is appended for it, and the run exits `1`. It is never reported as success (FR-006).
- **A batch fails as a whole** → the script degrades to per-path repair for that batch, because a batch failure does not say *which* path failed and FR-006 requires items to be listed individually.
- **Paths containing tabs or newlines** → survive intact: `find` emits three fixed numeric fields first and the path last, NUL-terminated, and `read` assigns the remainder verbatim to the last variable.
- **A crash between record and mutation** → the record already exists. Records are flushed to disk *before* the `chown` (FR-004b), so the recorded `outcome: "changed"` is the **intent**; if the mutation then fails, a corrective `failed` entry is appended for that path.
- **Concurrent downloads while repairing** → out of scope by construction: the repair blocks every download-writing service (FR-004g). Recorded in the contract so a later reader sees it was decided, not overlooked.
- **Host load** → the script re-execs itself under `nice -n 19 ionice -c 3` on first entry (guarded by `OWNERSHIP_REPAIR_RENICED` so it happens once, skipped entirely if either tool is absent). `exec` keeps the same pid, so a supervisor that captured the pid can still signal it.
- **Wiring, measured 2026-08-21T15:10Z:** `start.sh` calls this script from `run_ownership_gate()`, immediately after the ownership precondition passes, on the normal start path and on `--recreate`. A non-zero exit from the repair refuses the start; a missing script file also refuses the start (FR-004d — the repair blocks before any download-writing service accepts work). It is invoked under `nice -n 19 ionice -c 3` when both tools are present, which is belt and braces: this script also re-execs itself that way. The systemd unit `scripts/systemd/user/boba-stack.service` deliberately declares no `ExecStartPre` of its own — it runs `./start.sh --no-build` and inherits the single gate, so the two lifecycle paths cannot disagree about whether the repair ran. This wiring landed while this document was being written — it was measured directly in the working tree, not taken from the contract (§11.4.6). **Note the consequence:** because the marker makes a completed run a no-op, the per-start cost after the first successful pass is one scope read and one fingerprint comparison.

Internal behaviour

Output artifacts

Both live in `logs/ownership/` at the **project root** by default — operator-readable, on the host (never only inside a container, an E3 rule), and inside the already gitignored `logs/` tree so an

operational log never becomes a §11.4.30 versioned-artifact violation. `docs/qa/` was deliberately not used: that tree is curated QA evidence (§11.4.83) and is the wrong home for an operational log. The location is overridable per run via `--state-dir` / `OWNERSHIP_STATE_DIR`; the **default has not moved**.

`logs/ownership/repair-changes.ndjson` — the **E3 change record** of the run currently **in flight**. Appended to, one JSON object per line, greppable by path:


```
{"path":"/…/config/
        some.file","previous_uid":100999,"previous_gid":100999,"previous_mode":"64
```


`outcome` is one of `changed` | `skipped` | `failed`. For a declared path that does not exist there is no previous ownership state, so every ownership field is JSON `null` rather than a sentinel — `-1` is a plausible-looking uid and a later reader could not tell a sentinel from a real value.

This record is the operator's recovery trail for a repair they did not approve item by item. It holds **paths, uids, gids and modes only** — never file contents, never credential values (§11.4.10). `boba.db` and `.env` appear as paths; nothing of their contents ever does.

`logs/ownership/repair-changes.<UTC-timestamp>.ndjson` — the same record once its run **completed**. On a successful pass the in-flight record is rotated to a timestamped name and is never written again, so two completed runs can never share an artifact and one deletion can destroy at most one run's trail. A run that changed nothing produces no record at all (an empty file is removed rather than rotated, so a stream of empty files cannot bury the ones carrying evidence). An **interrupted** run's partial record is *not* rotated — it keeps the plain name and the resuming run appends to it, so one logical repair keeps one record.

What "durable" honestly means here (§11.4.6)

data-model E3 calls this record *durable* and *append-only*. Measured 2026-08-21, that overclaims. The honest statement is narrower:

- **Guaranteed** — the script only ever appends to the record of a run in flight. It never rewrites or truncates one, and never deletes a record belonging to a completed run.
- **Not guaranteed** — that the file still *exists* later. It lives in a gitignored `logs/` tree that repo tooling demonstrably cleans. On 2026-08-21 a marker and record written at 17:20 were both gone by 20:32 (`logs/` empty, dir mtime 19:21), deleted by a project actor, and nothing noticed. This script cannot defend a directory it does not own.

What it offers instead is **detection**: the marker names the record file of the run it marks, and a later run that finds that file gone prints

```
[ownership-repair] WARNING: the change record named by the
completion marker is MISSING: <name>
```

before opening a new record, so the loss is visible rather than absorbed. **Boundary:** if the whole state directory is removed, the marker goes with the record and there is nothing left to detect the loss against. Nothing here recovers a destroyed trail, and this document does not claim it does.

If you need a trail that survives repo tooling, point `--state-dir` somewhere outside the repository.

`logs/ownership/repair-marker.json` — the **E2 completion marker**:

```
{
  "completed_at": "2026-08-21T14:00:00Z",
  "scope_fingerprint": "<sha256 of the sorted declared scope>",
  "items_changed": 52,
  "record_file": "repair-changes.20260821T140000Z.ndjson"
}
```

Written via `mktemp` + `mv -f`, so a half-written marker can never be read by the next start. Validity is a literal match on the current fingerprint inside the file — a stale fingerprint is treated exactly as an absent marker.

`record_file` names this run's rotated change record, and is JSON `null` when the run changed nothing (no trail exists, so inventing a name would produce a false "MISSING" report on the next run). It is the *only* thing that makes a destroyed trail detectable.

The fingerprint is computed from the scope file's literal text **before** any repo-relative path is resolved, so it does not depend on the directory the run was started from — a change of directory must not invalidate a marker.

The pass

1. **Re-exec** under `nice`/`ionice` (once).
2. **Parse arguments**, resolve the operator `uid:gid` (`id -u` / `id -g` — the identity running the script, deliberately, because the feature's definition of "correct owner" is *whoever started the system*, not a configured constant that could drift).
3. **Read the scope.** Unreadable ⇒ exit `2`: reporting an unreadable scope as an empty scope would be the §11.4.201(6) false-null, where a blind instrument and a clean tree return the same quiet zero. Compute the fingerprint; failure ⇒ exit `2`.
4. **Order entries**, `preserve_mode` first (see *Overview*, point 4).

5. **Marker check** — skipped under `--force` or `--dry-run`.
6. **Open the change record** for append (skipped entirely under `--dry-run`).
7. **Per entry:** `find` the location for items whose uid **or** gid differs from the operator's, emitting `%U\t%G\t%m\t%p\0`. Non-recursive entries and regular files get `-maxdepth 0`. **This filter is the scope fence**: only items under a declared path are ever named, so an out-of-scope item cannot be reached even by accident (FR-005).
8. **Batch of 256:** append every record to the record file, **then** `chown` the batch, **then** restore mode bits where required. 256 is not a tuning knob — it is the width of the window in which records exist for items not yet mutated, traded against one fork per batch instead of one per item.
9. **Progress** is emitted as *items processed against items discovered*, throttled to once per second (FR-004e). A spinner or fixed-step estimate would not satisfy the requirement: it exists so a long run on a large library is distinguishable from a hang.
10. **Verdict.** All good ⇒ write the marker and exit `0`. Any failure ⇒ **no marker**, list the failures, exit `1`.

Related scripts

- `ownership_precondition.md` — `scripts/ownership_precondition.sh`, the startup check whose failure message points here. The precondition **detects**; this script **fixes**.
- `scripts/lib/ownership.sh` — the shared library both scripts source (`ownership_scope_entries`, `ownership_operator_uid`/`_gid`, `ownership_scope_fingerprint`, `probe_location`). Sourced rather than copied: a second copy would be the near-identical fork §11.4.251 forbids and would drift from the precondition and the gate that read the same scope.
- `scripts/pre_build/check_cm_ownership_invariants.sh` — invariant 33 (`CM-OWNERSHIP-INVARIANTS`), the standing regression gate that keeps the ownership routes from being reverted.
- `config/owned_paths.yaml` — the declared scope (E1). **Editing it changes the fingerprint and re-arms this repair.**
- `tests/unit/test_ownership_repair.sh` — the paired contract suite (golden-bad / golden-good / out-of-scope negative control / interrupt / preserve-mode cases).
- `tests/ownership/test_container_writes_owned_files.py` — the §11.4.115 RED that captured the original defect at uid 100999.
- `../guides/file-ownership.md` — the operator-facing narrative for the whole ownership feature.
- `../BOBA_DATABASE.md` — § 3, the backup procedure that this repair made performable again.

Cross-references

- `specs/002-user-owned-downloads/contracts/repair-cli.md` — the contract this script implements (FR-004, FR-004a–g, FR-005, FR-006, FR-015).
- `specs/002-user-owned-downloads/data-model.md` — **E1** scope, **E2** marker, **E3** change record.
- **§11.4.201** — assert the real condition; an unreadable scope is not an empty one, and a guard that refuses a healthy tree is as broken as one that passes a broken one.
- **§11.4.10** — the change record carries no credential values.
- **§11.4.30** — the record lives under the already-gitignored `logs/` tree.
- **§11.4.251** — why the scope/probe logic is sourced from one library rather than duplicated across the three consumers.
- **Principle XIII / §12.6** — the host runs mission-critical work, which is why the script re-execs itself under `nice -n 19 ionice -c 3`.

Anti-bluff & §11.4 discipline

- **Records precede mutation.** A record written *after* the `chown` is lost exactly when it matters — a crash mid-repair.
- **Failure is never rounded up to success.** Items that could not be repaired are listed individually, recorded with `outcome: "failed"`, and the run exits `1` with no marker (FR-006).
- **The marker is earned, not assumed.** It is written only after a fully successful pass, so "already done" can never be claimed about work that was not performed — including work that a *scope change* newly declared.
- **Permission bits are never widened.** `preserve_mode` and special-bit items have their original mode restored after `chown`.
- **The scope fence is structural.** `find` is rooted at declared paths and `chown -h` refuses to follow symlinks out, so out-of-scope reach is impossible by construction rather than by intention.
- §11.4.263: this script signals no processes — there is no `kill`, `pkill`, or `killpg` anywhere in it, so the `pgid <= 1` broadcast-kill hazard cannot arise.

Last verified

2026-08-21 — every flag, exit code, environment variable, record field, and edge case in this document was read from `scripts/ownership_repair.sh` and `scripts/lib/ownership.sh`, not from the script's usage text alone. The automatic first-start invocation is

described from the call actually present in the working tree at 2026-08-21T15:10Z (`start.sh` → `run_ownership_gate()`), not from the contract's word for it.

2026-08-21 (Revision 2) — re-verified after four behaviour changes, each with a test-first RED observed in `tests/unit/test_ownership_repair.sh` before the fix:

- repo-relative declared paths are now resolved against the project root (Case 14). Measured pre-fix: a run started from `/` reported `FAILED fixture/... — declared path does not exist and is not optional` and exited 1 on a path that plainly existed — the §11.4.201(1) false-positive refusal. The sibling `scripts/ownership_precondition.sh` already absolutised, so the two readers of one scope file had disagreed about relative entries.
- the state directory is overridable, default unchanged (Case 13);
- a completed run's record is rotated to its own artifact (Case 12);
- a destroyed record named by the marker is reported (Case 11).

Suite: 34 assertions passing before this change, 44 after, 0 failed, 0 skipped.

2026-08-25 (Revision 3) — re-verified after the T028 independent review returned NO-GO (0 blocking, 2 important, 3 minor, 2 nit). Every remediation captured a RED in `tests/unit/test_ownership_repair.sh` against the pre-fix artifact before its fix existed, and every new test was proven to catch its own negation by a paired §1.1 mutation:

- **an empty scope is no longer a completed repair** (Case 16). Measured pre-fix: `paths: []`, a mistyped top-level key, and an unset `${VAR}` each exited `0` and wrote a marker with the fingerprint of the empty string.
- **the declared path is itself fenced** (Case 17). Measured pre-fix: the shipped `${QBITTORRENT_DATA_DIR:-/mnt/DATA}` entry with the variable repointed launched `find /` over the whole filesystem — observed in `ps` and killed by pid. `..` was never resolved, and `absolutise("/")` returned `/`.
- **the hardlink escape is documented instead of denied** (Case 18) — the header had claimed out-of-scope reach was impossible by construction; it is not, and the correction is machine-checked against the measured behaviour.
- **a failed `chown` prints its reason** (Case 19); **a failed mode restore is reported and blocks the marker** (Case 20); **the fingerprint comment describes the environment-expanded parse it is actually computed from** (Case 21); **a failure on the declared root is diagnosed as such** (Case 22).

Suite: 44 assertions passing before this change, 86 after, 0 failed, 0 skipped. The live shipped scope was re-checked against the new fence with the real `.env`: all six declared entries ACCEPT, and the scope fingerprint is byte-identical to the pre-change one, so no existing marker is invalidated.